Worksheet 10 — API Security (3 hrs)

> **Course:** Software Security (KOSEN69) · Week 10 **Aligned:** OWASP API Security Top 10:2023 — API1 BOLA · API3 Broken Object Property Level Auth (Mass Assignment) · API4 Unrestricted Resource Consumption **Signature game:** 🦹 crAPI Raid
>
> **Ethics note:** Run exploits **only** against the lab targets shipped here (`vulnerable_api.py` on `:8080`) or your own OWASP crAPI instance. Never test BOLA, mass assignment, or brute-force against systems you do not own or are not explicitly authorized to test.

Part 1 — Student Information

Name	Student ID	Date	Group
Supichaya Rabiabnaweenurak	6631503118	23/10/2569	---

Part 2 — Lecture Questions

1. Define **BOLA (API1:2023)**. Why is it the #1 API risk, and why can't a WAF reliably stop it?
2. What is **mass assignment / broken object property level authorization (API3:2023)**? Contrast a blocklist vs. an allow-list of bindable fields.
3. **API4:2023** is unrestricted resource consumption. Give two distinct impacts (security and availability) of an unthrottled `/api/login`.
4. In `vulnerable_api.py`, `current_user()` trusts the `X-User-Id` header. Why is a client-supplied header NOT authentication, and what should replace it?
5. Why does object-level authorization have to be checked **per request, per object** rather than once at login?

Answers

1. **BOLA (API1:2023)** occurs when an API authenticates or identifies a caller but does not verify that the caller is allowed to access the specific object requested. It is ranked as the top API risk because APIs expose many object identifiers and one missing ownership check can expose large amounts of personal or business data. A WAF cannot reliably stop it because the request is syntactically valid and the difference between an authorized and unauthorized object id requires application-level context.

2. **Mass assignment** happens when the server binds every client-supplied field to an object, including sensitive fields such as `is_admin` or `balance`; this is part of API3:2023 Broken Object Property Level Authorization. A blocklist rejects only known-dangerous fields and can fail when a new sensitive field is added. An allow-list accepts only explicitly approved fields, such as `username` and `password`, while the server sets privileged properties itself.

3. An unthrottled `/api/login` allows attackers to make unlimited password guesses, increasing the chance of account takeover. It can also consume CPU, database connections, and network capacity, causing slow responses or denial of service. Rate limiting, monitoring, and account/IP-based controls reduce both impacts.

4. A client can send any `X-User-Id` value, so the header is only an untrusted claim and does not prove the caller's identity. It should be replaced by a verified session or signed access token created after authentication; the server must derive the user identity from that trusted credential.

5. Authorization must be checked on every request because a user can request a different object after login, and permissions or ownership can change during the session. The server therefore has to compare the authenticated caller with the specific object on each access, rather than treating login as permission to read every object.

Part 3 — Hands-on Lab (145 min)

Learning goals: Exploit BOLA, mass assignment, and a missing rate limit against a live API; then read the secure version and explain each fix.

Prerequisites: Docker + Docker Compose, `curl`. (Optional bonus: cloned OWASP crAPI.)

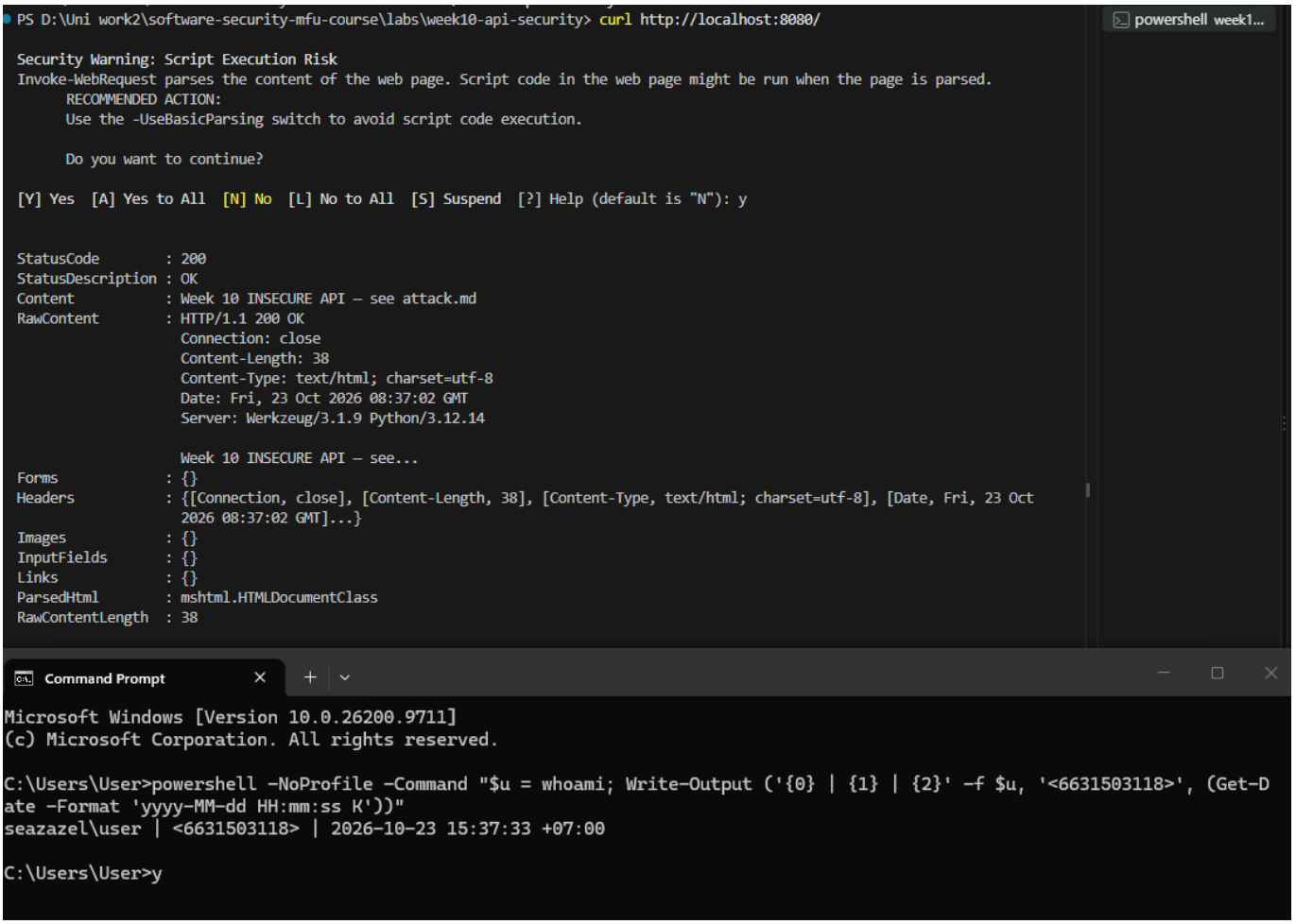
Environment setup

```
cd labs/week10-api-security
docker compose up         # INSECURE API on :8080, SECURE API on :8081
# Seeded users: alice(id 1) bob(id 2) carol(id 3, admin, balance 9999)
```

What to submit per task: the exact command(s) + raw response/HTTP status, a screenshot, and a 2–3 sentence mitigation note mapping the bug to its OWASP API id.

Task 0 — Onboarding (15 min)

Confirm both APIs respond: `curl http://localhost:8080/` and `curl http://localhost:8081/`. Note which port is insecure. Record the three seeded users. **Deliverable:** both root responses + the user table.



```
PS D:\Uni work2\software-security-mfu-course\labs\week10-api-security> curl http://localhost:8080/

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.

      Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): y


StatusCode        : 200
StatusDescription : OK
Content           : Week 10 INSECURE API — see attack.md
RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 38
                    Content-Type: text/html; charset=utf-8
                    Date: Fri, 23 Oct 2026 08:37:02 GMT
                    Server: Werkzeug/3.1.9 Python/3.12.14

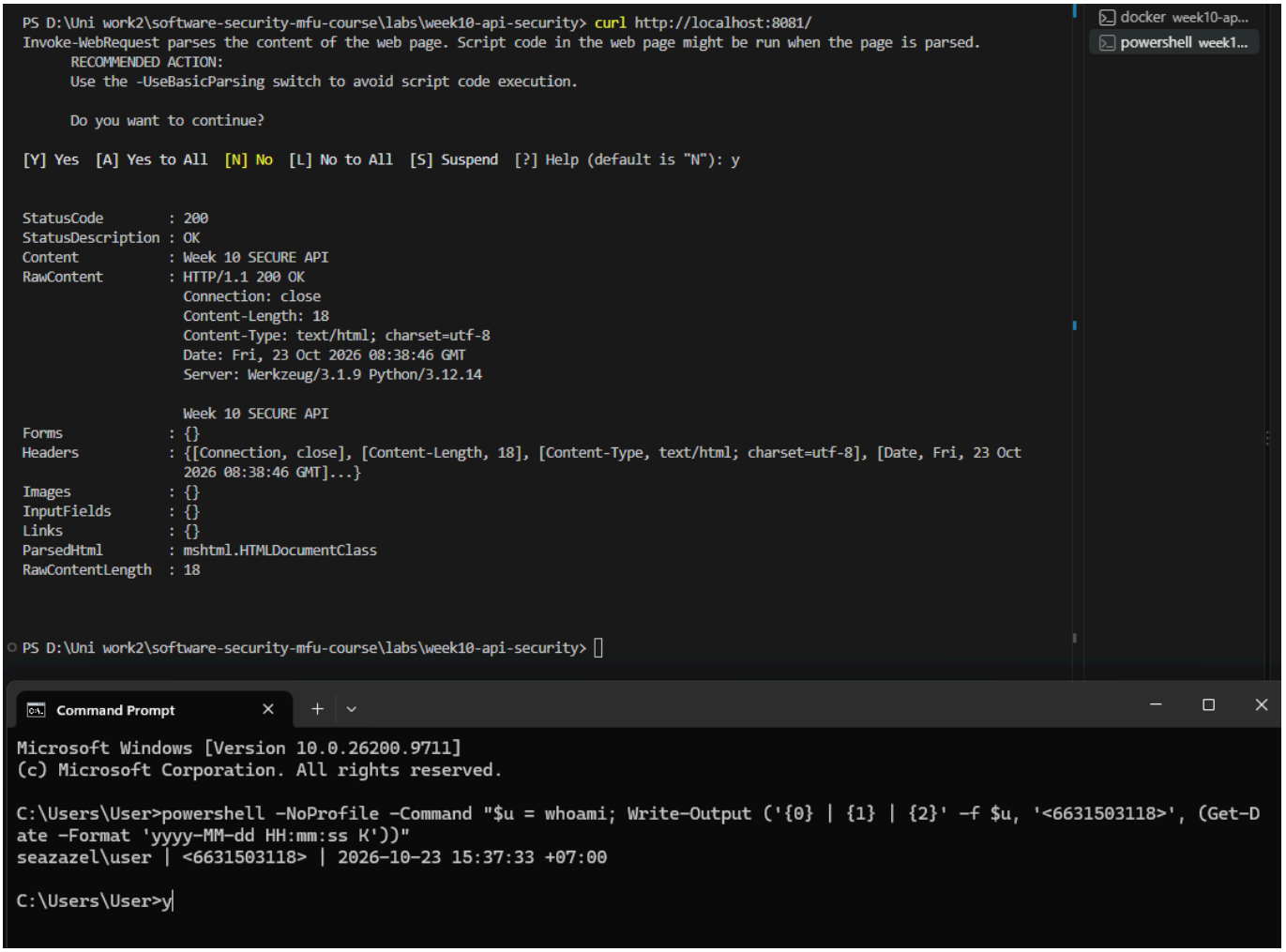
                    Week 10 INSECURE API — see...
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 38], [Content-Type, text/html; charset=utf-8], [Date, Fri, 23 Oct
                    2026 08:37:02 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 38
```



```
Microsoft Windows [Version 10.0.26200.9711]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User>powershell -NoProfile -Command "$u = whoami; Write-Output ('{0} | {1} | {2}' -f $u, '<6631503118>', (Get-D
ate -Format 'yyyy-MM-dd HH:mm:ss K'))"
seazazel\user | <6631503118> | 2026-10-23 15:37:33 +07:00

C:\Users\User>y
```



```
PS D:\Uni work2\software-security-mfu-course\labs\week10-api-security> curl http://localhost:8081/
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.

      Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): y


StatusCode        : 200
StatusDescription : OK
Content           : Week 10 SECURE API
RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 18
                    Content-Type: text/html; charset=utf-8
                    Date: Fri, 23 Oct 2026 08:38:46 GMT
                    Server: Werkzeug/3.1.9 Python/3.12.14

                    Week 10 SECURE API
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 18], [Content-Type, text/html; charset=utf-8], [Date, Fri, 23 Oct
                    2026 08:38:46 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 18

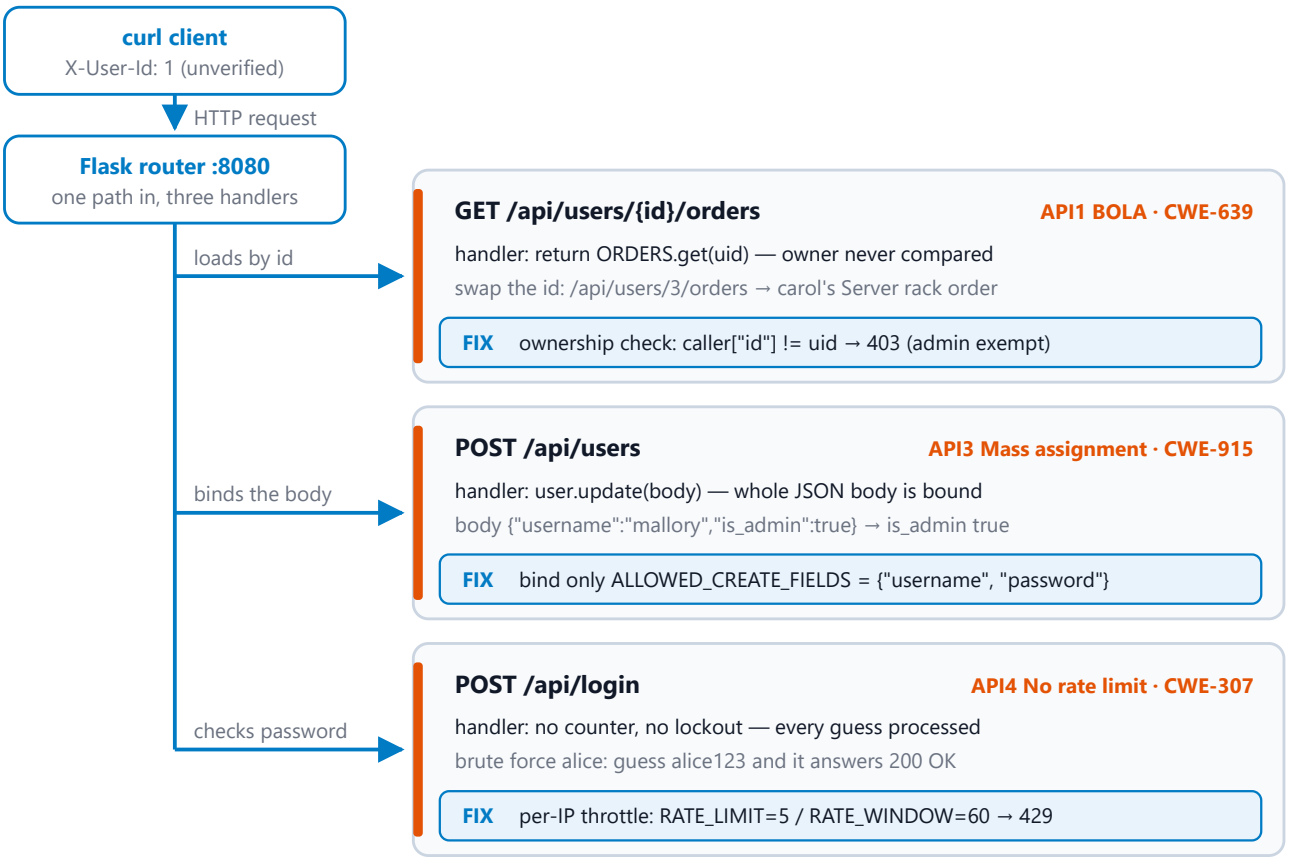

PS D:\Uni work2\software-security-mfu-course\labs\week10-api-security>
```



```
Microsoft Windows [Version 10.0.26200.9711]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User>powershell -NoProfile -Command "$u = whoami; Write-Output ('{0} | {1} | {2}' -f $u, '<6631503118>', (Get-D
ate -Format 'yyyy-MM-dd HH:mm:ss K'))"
seazazel\user | <6631503118> | 2026-10-23 15:37:33 +07:00

C:\Users\User>y
```



All three are authorization and design flaws, not input flaws.
The JSON is well-formed: escaping and encoding change nothing.
A WAF cannot know which order id belongs to which caller.

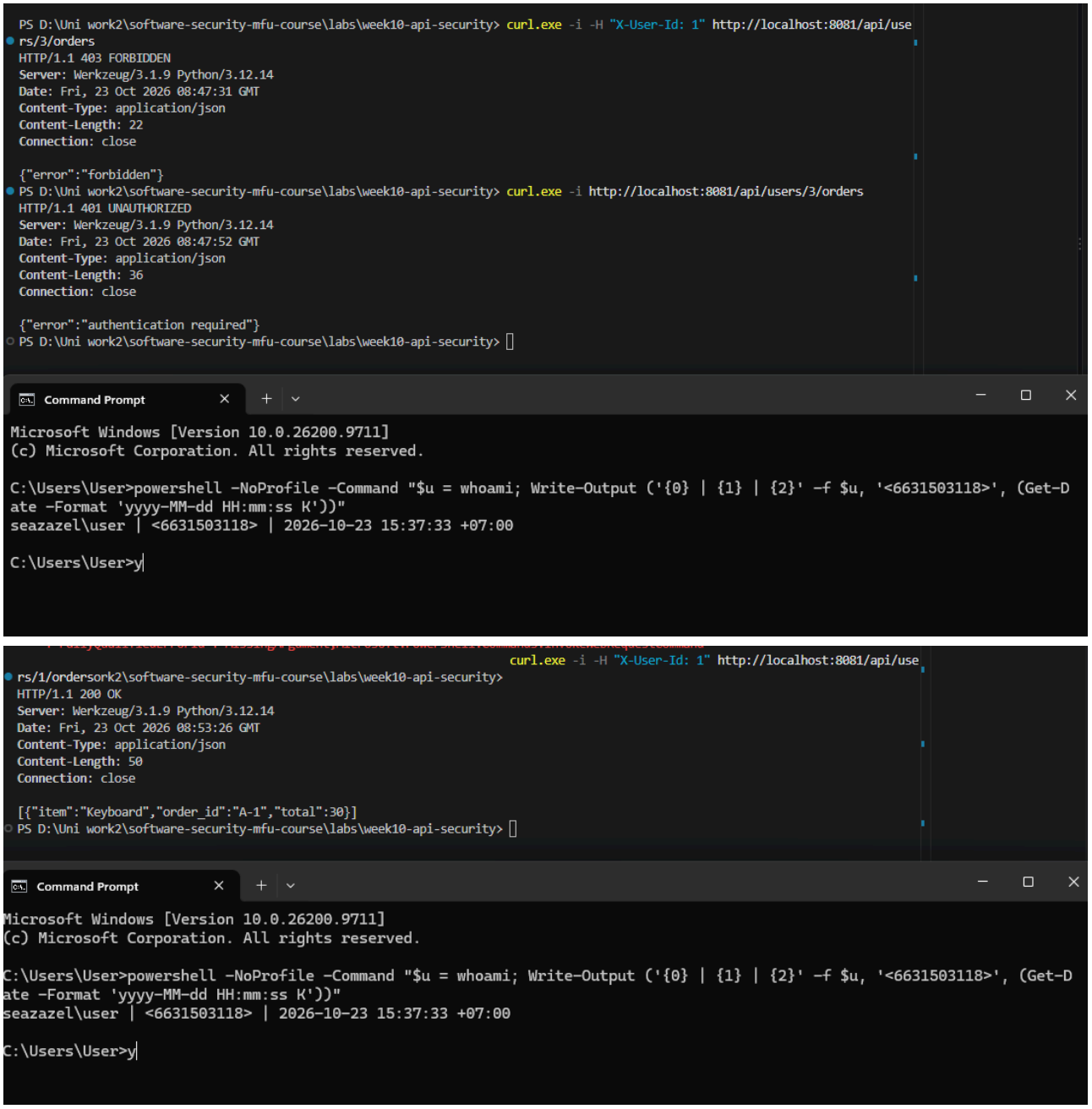
Task 1 — BOLA: read another user's orders (35 min)

Goal: Read carol's (admin) orders without being carol — API1:2023. **Steps:**

1. `curl http://localhost:8080/api/users/3/orders` — note you receive the "Server rack" order with no auth.
2. Iterate the id: `curl http://localhost:8080/api/users/2/orders`.
3. On the secure API observe the ladder: `curl -i http://localhost:8081/api/users/3/orders` (401), `curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/3/orders` (403), `curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/1/orders` (200). **Deliverable:** the leaked order JSON + the 401/403/200 transcript + mitigation note.

- [{"item":"Server rack","order_id":"C-1","total":5000},{"item":"FLAG{bola_demo}","order_id":"C-FLAG","total":0}]

- [{"item":"Mouse","order_id":"B-1","total":15}]

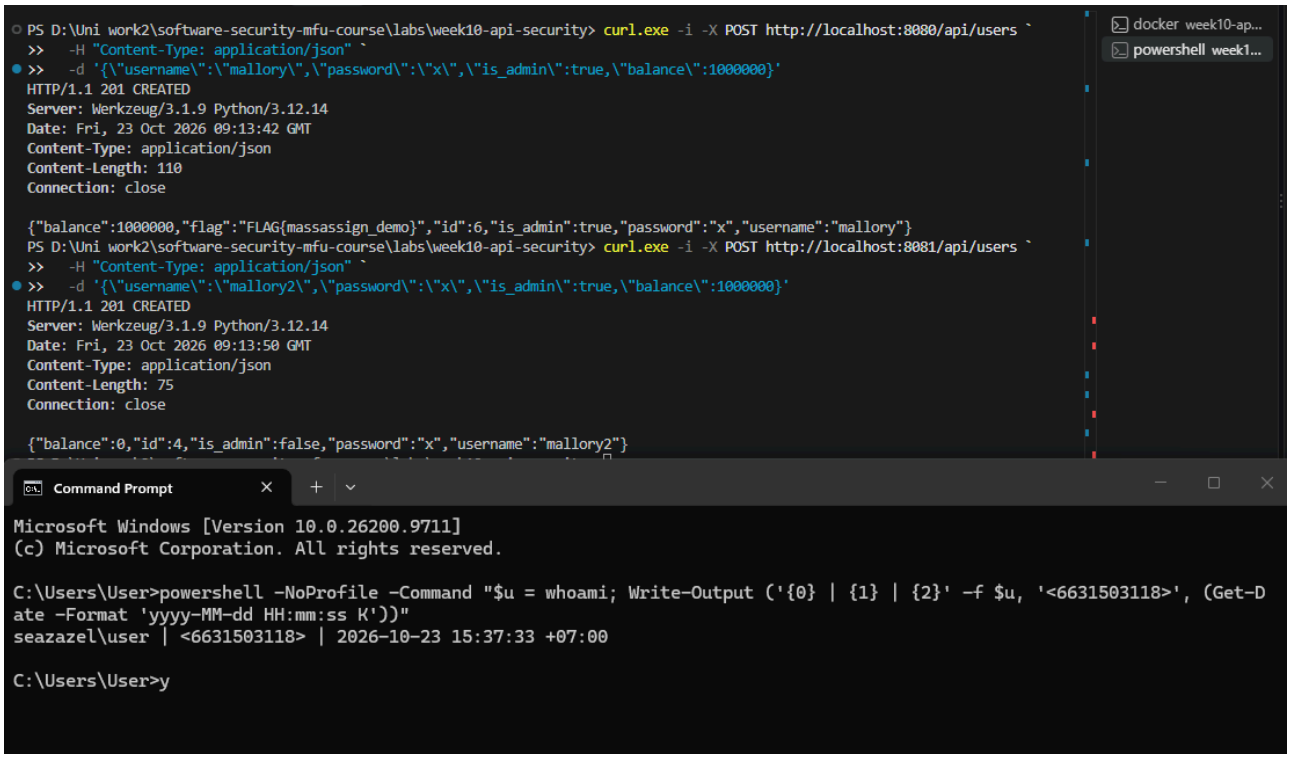


> - This is OWASP API1:2023 BOLA because the server must verify that the authenticated user owns the requested order before returning it. The secure API performs a per-object ownership check and returns 403 Forbidden for another user's order; in production, X-User-Id should be replaced with a verified session or access token.

Task 2 — Mass assignment: self-promote to admin (30 min)

Goal: Smuggle `is_admin` + `balance` into user creation — API3:2023. **Steps:**

1. `curl -X POST http://localhost:8080/api/users -H "Content-Type: application/json" -d '{"username":"mallory","password":"x","is_admin":true,"balance":1000000}'`
2. Confirm the response echoes `"is_admin": true, "balance": 1000000`.
3. Repeat against `:8081` and confirm the smuggled fields are forced to `is_admin:false, balance:0`.
   Deliverable: both responses side by side + mitigation note (allow-list in `solution_api.py`).

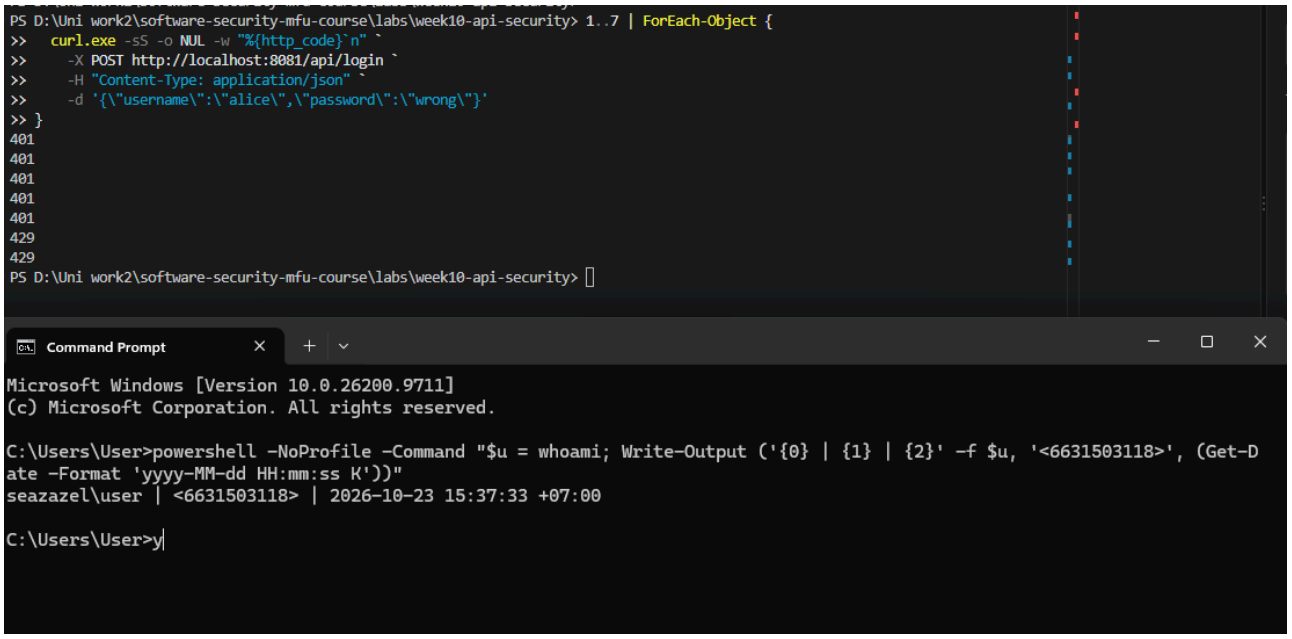


> - This is OWASP API3:2023 Broken Object Property Level Authorization, specifically mass assignment. The vulnerable API accepts every field supplied by the client, allowing attackers to set is_admin and balance. The secure API uses an allow-list containing only username and password, while the server sets is_admin to false and balance to 0.

Task 3 — Unrestricted resource consumption: brute-force login (25 min)

Goal: Show `/api/login` has no throttle — API4:2023. **Steps:**

1. Run the brute-force loop from `attack.md` against `:8080` (guesses incl. `alice123`) — all attempts processed.
2. On `:8081` loop 7 times: `for i in $(seq 1 7); do curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8081/api/login -H "Content-Type: application/json" -d '{"username":"alice","password":"wrong"}'; done`
3. Confirm the 6th/7th attempt returns `429`. **Deliverable:** the `401 401 401 401 401 429 429` sequence + mitigation note.



> - This is OWASP API4:2023 Unrestricted Resource Consumption. The vulnerable login endpoint accepts unlimited password guesses, enabling brute-force attacks and consuming server resources. The secure API applies a per-IP limit of five attempts within 60 seconds and returns 429 Too Many Requests afterward; production systems should also use account-based throttling, monitoring, and gradual lockouts.

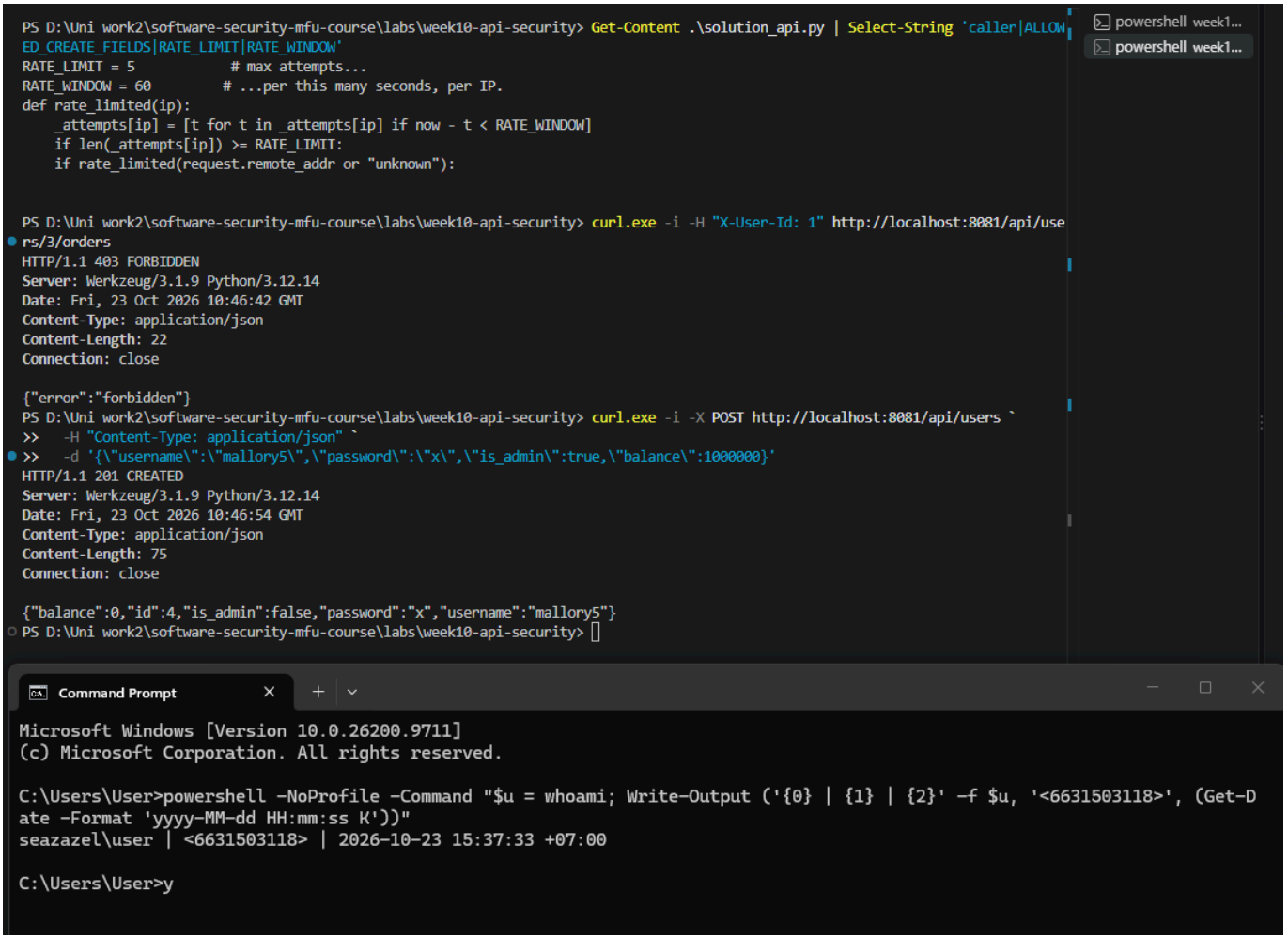
Task 4 — Bonus: crAPI Raid (optional, 20 min)

Against your own OWASP crAPI instance (`git clone https://github.com/OWASP/crAPI.git`), capture one BOLA or mass-assignment flag. **Deliverable:** flag + endpoint + which API Top 10 id it maps to.

Task 5 — Defend / fix it (20 min)

Goal: Explain the fixes using the secure reference `solution_api.py`. **Steps:** Read the three `# --- FIX ...` blocks. For each, quote the line(s) that defeat your Task 1–3 exploit: the per-object ownership check (`caller["id"] != uid and not caller["is_admin"]`), `ALLOWED_CREATE_FIELDS` binding, and the `RATE_LIMIT=5 / RATE_WINDOW=60` limiter. **Deliverable:** for each of API1/API3/API4 — the exploit that

worked on `:8080`, the line in `solution_api.py` that blocks it, and the new HTTP status on `:8081`.



Part 4 — Reflection

1. **Mapping:** complete a table — finding | endpoint | OWASP API id | fix applied.
2. **Real breach:** research an API authorization breach (e.g., the Optus or T-Mobile BOLA/IDOR incidents). Which API Top 10 id matches, and would the `solution_api.py` ownership check have prevented it?
3. **Best mitigation:** of object-level auth, allow-list binding, and rate limiting — which gives the most risk reduction for an API platform, and why?

Reflection answers

Finding	Endpoint	OWASP API id	Fix applied
BOLA: read another user's orders	`GET /api/users/{id}/orders`	API1:2023	Authenticate the caller and check object ownership for every request; allow admins explicitly.
Mass assignment	`POST /api/users`	API3:2023	Bind only `username` and `password`; set `id`, `balance`, and `is_admin` on the server.
Brute-force/resource exhaustion	`POST /api/login`	API4:2023	Limit each IP to five attempts per 60-second window and return `429` afterward.

The 2022 Optus breach is relevant because attackers were able to enumerate identifiers through an exposed API and access customer records that were not properly authorized for them. This maps to API1:2023 BOLA. A per-object ownership check like the one in `solution_api.py` would have blocked access to records that did not belong to the authenticated caller, although the real system would also need strong authentication and controls against identifier enumeration.

Object-level authorization gives the greatest risk reduction because it directly protects every user's data from cross-user access, which is the highest-impact flaw in this lab. Allow-list binding and rate limiting are still necessary: the first prevents privilege changes, while the second limits password guessing and resource exhaustion. These controls are complementary rather than interchangeable.

Grading rubric (100)

Criterion	Weight
Lecture questions (Part 2)	20
Exploitation + evidence (Tasks 1–4: commands, output, screenshots)	40
Defense (Task 5: fixes mapped to `solution_api.py`)	25
Reflection (Part 4: mapping, breach, best mitigation)	15
Total	**100**

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*
    1. What did you do, and **why did the vulnerability work**?
    2. **Why does your fix actually stop it** — and what could still break it?

Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

Audit response

The AI suggested using escaped JSON such as `-d '{\"username\":\"alice\",\"password\":\"wrong\"}'` for PowerShell. That advice was useful for this environment, but it initially assumed the ordinary Unix `curl` command would work unchanged in PowerShell and did not explain that `curl` is an alias for `Invoke-WebRequest`. The first unescaped request therefore produced `400 Bad Request` because the server received invalid JSON, not `401` for bad credentials. I verified the corrected command against the local secure API and obtained the expected `401` responses followed by `429` responses.

The secure fix itself is also not a complete production authentication design: `solution_api.py` still treats the client-controlled `X-User-Id` header as identity. The ownership check correctly demonstrates API1, but a real deployment must derive the caller from a verified session or access token before applying the object-level authorization decision.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*

Comprehension answers

A. Explain in Plain English: The vulnerable orders endpoint returns the order list for whichever user id appears in the URL, without checking who made the request or whether that user owns the object. An attacker can therefore change `/users/1/` to `/users/3/` and receive another user's orders; the secure version checks the caller and returns `403` for an unauthorized object.

B. Final prompt:

> Fix the `GET /api/users/<int:uid>/orders` handler in this Flask lab for OWASP API1:2023 BOLA. Require a verified authenticated caller, deny unauthenticated requests with HTTP 401, allow a caller to read only their own user's orders unless they are an admin, and deny other objects with HTTP 403. Preserve the existing response format and do not trust a URL id or client-supplied privilege fields as proof of ownership. Provide the minimal code change and explain how to test ids 1 and 3.

Verified result: With no identity, the secure endpoint returned `401`. With `X-User-Id: 1` requesting user 3's orders, it returned `403`; with the same identity requesting user 1's orders, it returned `200`. This confirms that the ownership check blocks the exploit while preserving authorized access.